

بسمه تعالی

آزمایش هشتم آزمایشگاه معماری کامپیوتر

اضافه کردن دستورات پرش و حافظه به کامپیوتر از 7

روزبه معینی 401106544

احسان محترم 401106458

دانشگاه صنعتی شریف

بهار 1405

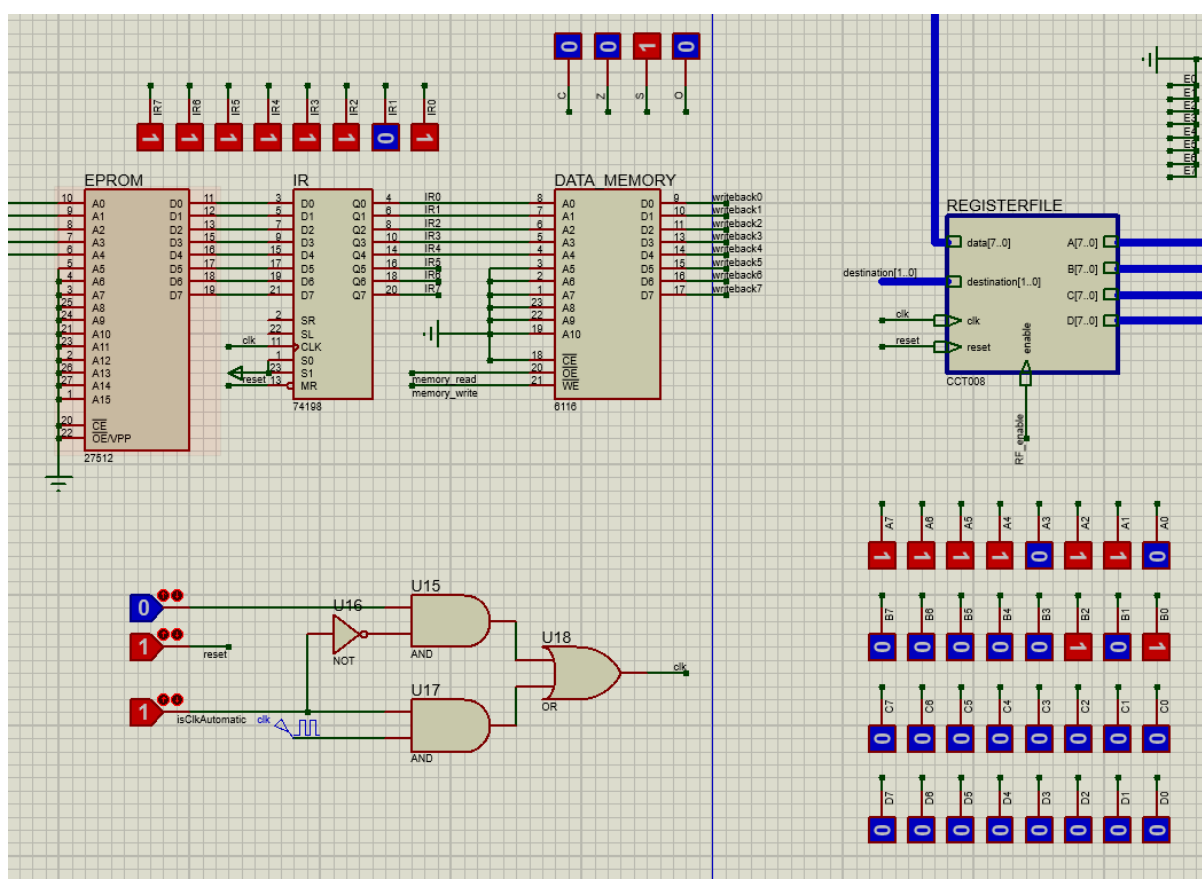
فهرست

۱. مقدمه	3
۲. طراحی داخل proteus	4
ساختار کلی مدار	4
زیربخش‌ها	4
تغییرات بخش REGTSEFILE	5
تغییرات بخش ALU	5
ماژول جدید PC و رجیستر IR	7
حافظه دیتا	8
ماژول جدید Control	8
۳. بررسی و تست در شبیه‌سازی	11
تست اول: فیبوناچی	11
تست دوم: جمع دو عدد بزرگتر از 8 بیت با رقم نقلی	13
تست سوم: جمع دو عدد 32 بیتی ساده	15

۱. مقدمه

هدف از این گزارش، مستندسازی مراحل انجام شده در آزمایش شماره ۸ آزمایشگاه معماری کامپیوتر است. در این تمرین، به افزودن قابلیت پرش شرطی و دسترسی به حافظه به مدار آزمایش قبلی می‌پردازیم.

نحوه دادن ورودی در تصویر ۱ مشخص شده است. به دو نحوه دستی و اتوماتیک با تنظیم سیگنال isClkAutomatic شروع به دادن کلاک به سیستم می‌کنیم و برنامه یکی یکی دستورات را از ۳۲ بیت دستور ذخیره شده در EPROM می‌خواند و اجرا می‌کند. برای ورودی دادن به حافظه EPROM نیز یک ایمپچ از ۳ ایمپچ فراهم شده برای تست را انتخاب و با دادن کلاک برنامه را اجرا می‌کنیم. با استفاده از قابلیت‌های دیباگر نیز می‌توانیم با توقف شبیه‌سازی در مرحله دلخواه، محتویات ذخیره شده در رم را مشاهده کنیم. تعدادی نمایشگر وضعیت برای مشاهده نحوه پیشرفت کار اضافه شده است. ۸ بیت سمت چپ مقدار دستوری که فچ شده است را نشان می‌دهد. ۴ پین کناری مقادیر کنونی فلگ‌ها را نشان می‌دهد و ۴ مقدار ۸ بیتی پایین تصویر مقادیر رجیسترهاست.

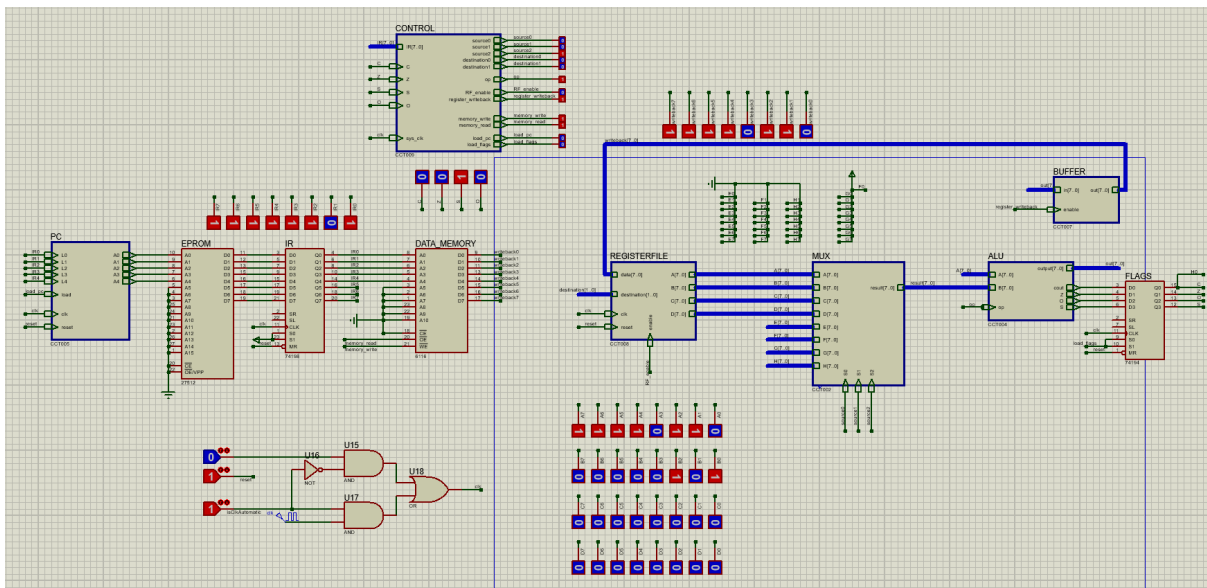


شکل ۱. ساختار ورودی و نمایشگرهای وضعیت

۲. طراحی داخلی proteus

ساختار کلی مدار

شمای کلی مدار در شکل 2 آمده است. در مجموع کل پیاده سازی در سه بخش جدید و تعدادی تغییر نسبت به طراحی آزمایش 6 خلاصه می شود. یک بخش کنترل برای تولید سیگنال های کنترلی بر اساس دستور و فلگ ها، یک بخش برای محاسبات PC و دسترسی آن به حافظه EPROM و یک حافظه دیتا از نوع SRAM. برخی جزئیات پیاده سازی نیز در مازول های قدیمی تغییر یافته است.



شکل 2. ساختار کلی

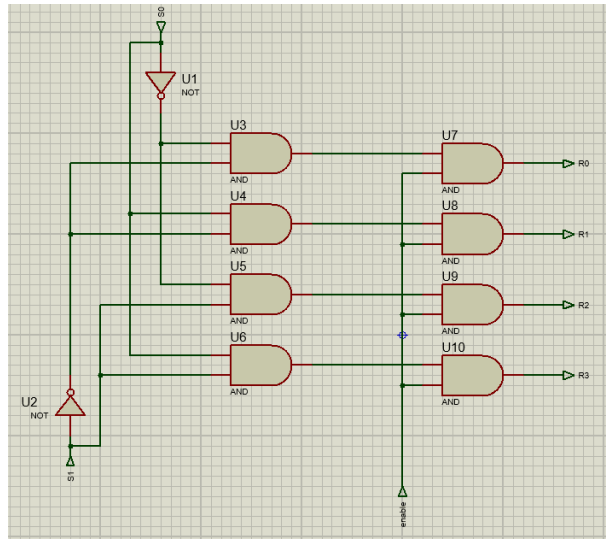
زیربخش ها

تغییرات مدار را به 5 زیربخش کوچک تر مطابق دستور کار شکسته ایم که در شکل 2 آن را مشاهده کردید. به ترتیب تغییرات و مازول های جدید عبارتند از:

- تغییرات جزئی در رجیسترفایل
- تغییرات ALU شامل تولید پرچم ها و افزودن رجیستر نگهدارنده پرچم ها. همچنین افزودن مازول جدید buffer که یک بافر سه وضعیتی برای نگهداری خروجی ALU و ذخیره آن در صورت نیاز است.
- مازول جدید PC و رجیستر IR: شمارنده با قابلیت لود برای پیاده سازی دستور پرش
- اضافه شدن حافظه دیتا
- مازول جدید کنترل: تولیدکننده سیگنال های کنترلی

تغییرات بخش REGITSERFILE

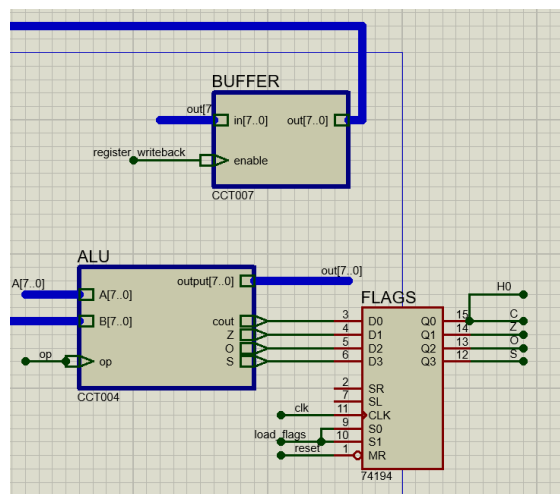
یک سیگنال enable به رجیسترفایل اضافه شده است. این سیگنال در صورت صفر بودن هر 4 خروجی دیکودر را غیرفعال می‌کند تا عمل نوشتن انجام نشود. این سیگنال به ما کمک می‌کند در دستوراتی که مقادیر رجیسترها باید تغییر نکنند، هیچکدام را تغییر ندهیم. طراحی جدید بخش دیکودر را در شکل مشاهده می‌کنید. (شکل 3)



شکل 3. افزودن سیگنال enable به دیکودر رجیسترفایل

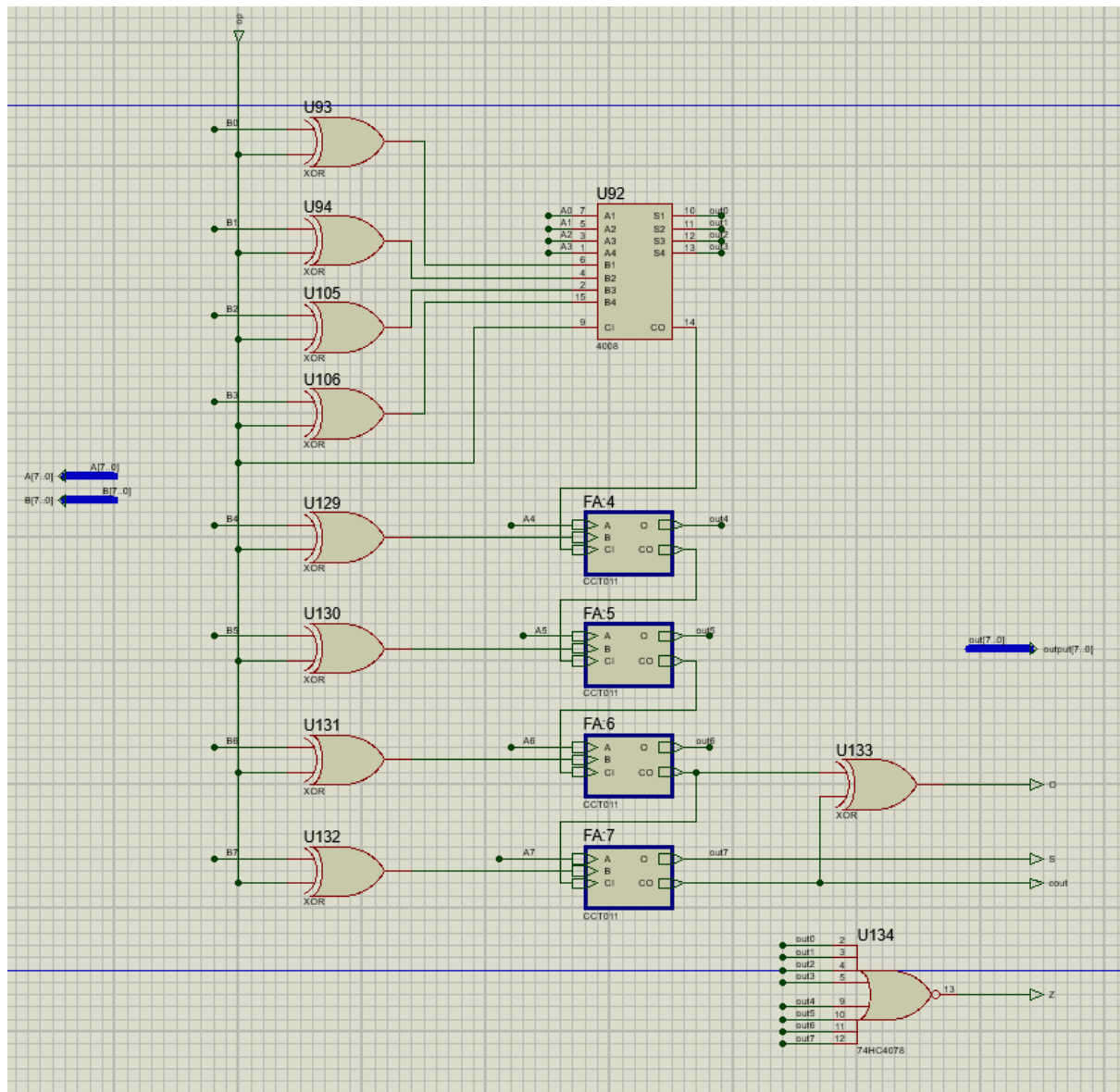
تغییرات بخش ALU

برای این بخش یک رجیستر 74194 برای نگهداری پرچم‌های خروجی محاسبات از جمله سرریز، نقلی، صفر و بیت علامت افزوده‌ایم. همچنین یک tri state buffer داریم که با سیگنال کنترلی writeback فعال می‌شود و خروجی ALU را به ورودی رجیسترفایل متصل می‌کند. (شکل 4-1)



شکل 4-1. تغییرات واحد محاسبات

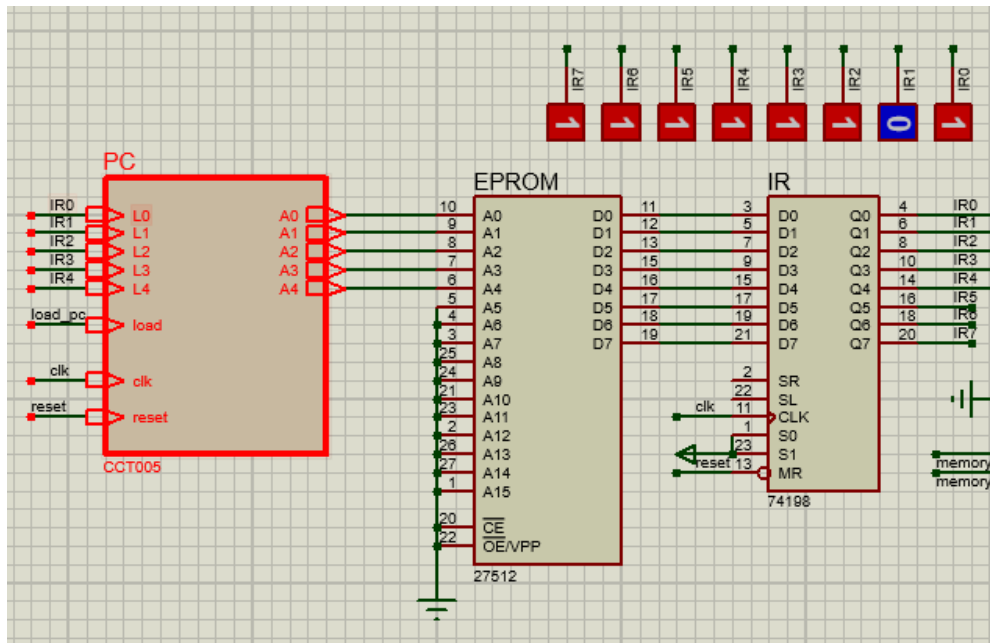
در تغییرات داخل خود ALU، نیاز به تولید پرچم‌های سرریز، نقلی، صفر و بیت علامت داریم. به علت اینکه برای محاسبه سرریز به XOR دو رقم نقلی آخر نیاز داریم، به جای دومین عدد جمع‌کننده 4008، 4 عدد تمام جمع‌کننده قرار دادیم. همچنان همه بیت‌های خروجی را برای تولید پرچم صفر با هم NOR کردیم. (شکل 2-4)



شکل 2-4. جایگزینی 4008 با FA 4 برای محاسبه پرچم‌ها

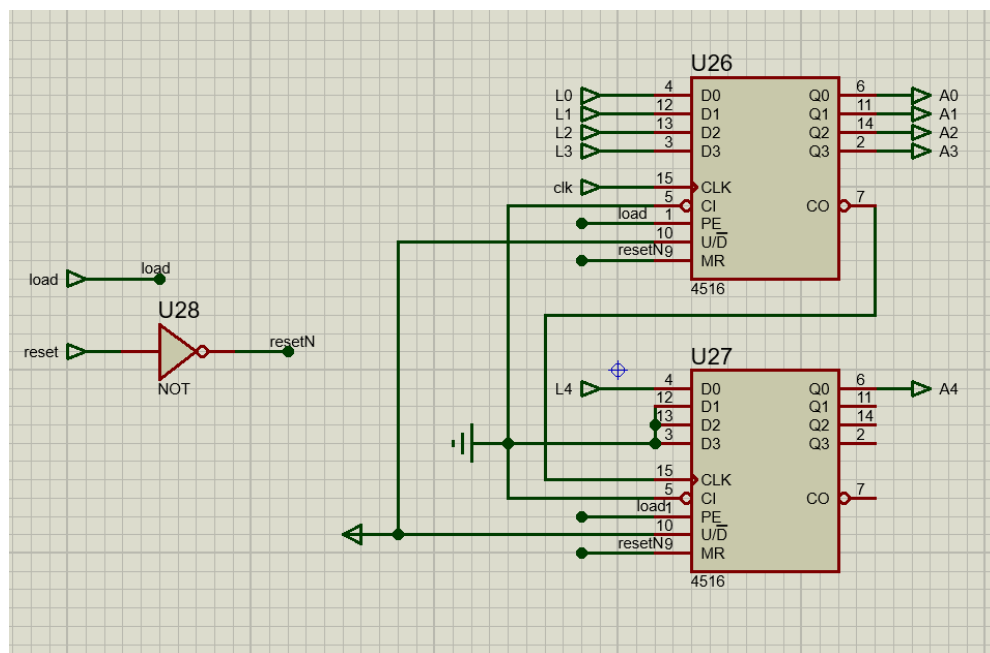
ماژول جدید PC و رجیستر IR

ساختار کلی بدین صورت است که واحد PC به یک EPROM از نوع 27512 متصل می‌شود، دستور از روی آن خوانده می‌شود و روی رجیستر IR نوشته می‌شود. (شکل 5-1)



شکل 5-1. ساختار واحد جریان برنامه

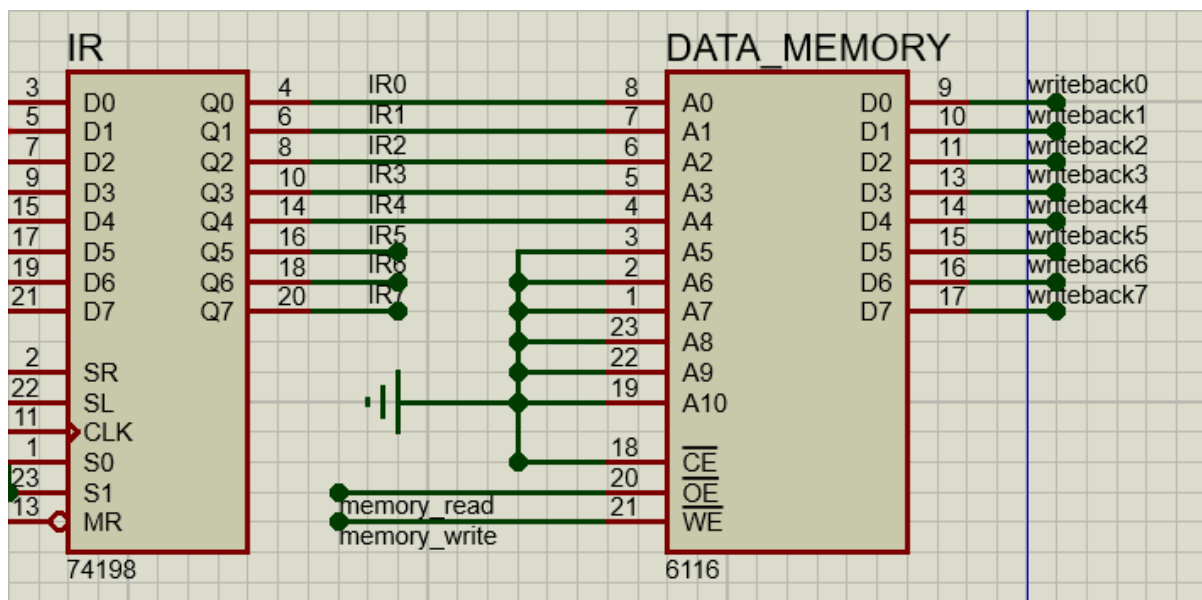
ساختار داخل PC نیز شامل دو شمارنده از نوع 4516 است که قابلیت لود دارند. در صورت فعال شدن سیگنال کنترلی load_pc، مقدار جدید در PC لود می‌شود (شکل 5-1)



شکل 5-2. پیاده‌سازی PC با دو 4516

حافظه دیتا

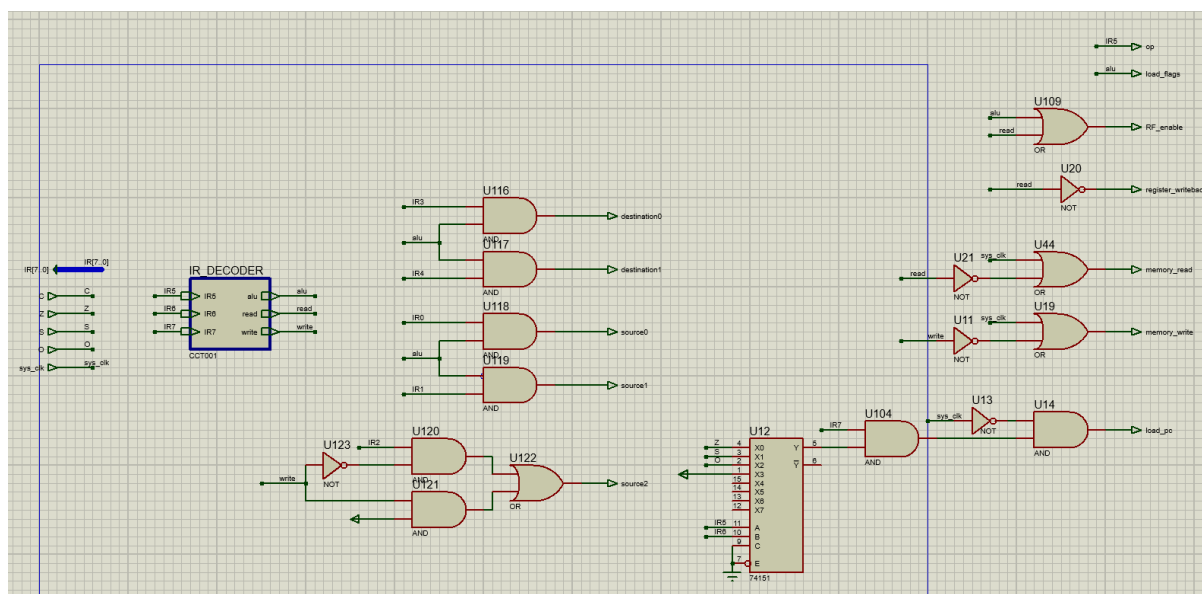
از یک آی سی 6116 که یک SRAM است استفاده می‌کنیم و 5 بیت کم ارزش دستور را به عنوان آدرس و دو بیت کنترلی خواندن و نوشتن را به آن وصل می‌کنیم. همچنین خروجی آن را به باس ورودی رجیسترفایل متصل می‌کنیم تا اتصالات حافظه کامل شود. (شکل 6)



شکل 6. پیاده‌سازی حافظه

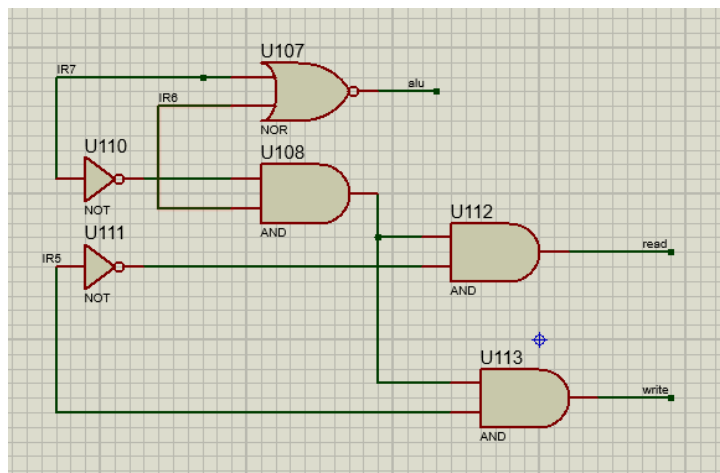
ماژول جدید Control

این بخش کنترل کل سیستم را برعهده دارد. (شکل 7-1)



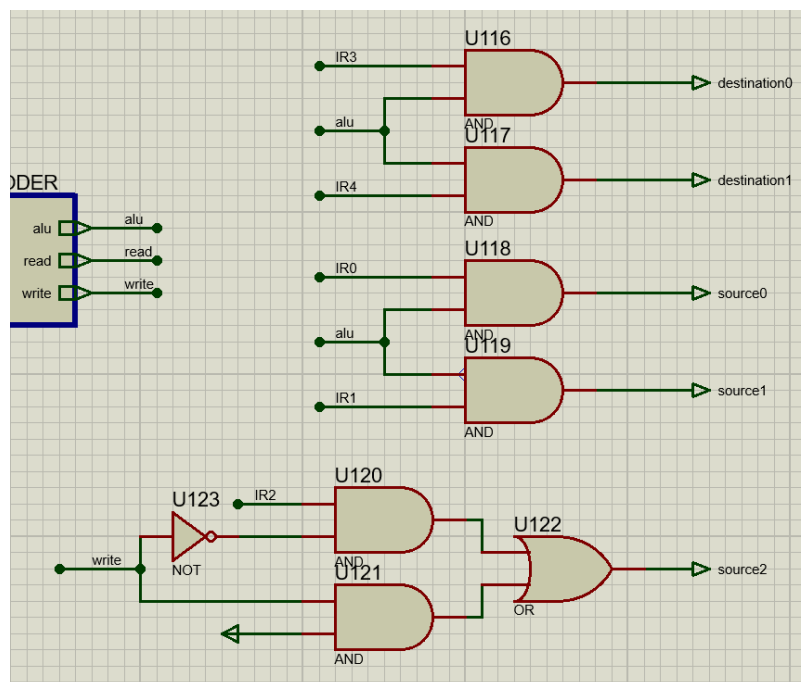
شکل 7-1. پیاده‌سازی بخش کنترل

اول از روی 3 بیت پرارزش دستور، مشخص می‌کنیم دستور از نوع عملیاتی، دسترسی به حافظه یا پرش است. اینکار را دیکودر برعهده دارد. alu به معنای دستور محاسباتی، read , write یعنی دستور خواندن و نوشتن حافظه و در نهایت خود بیت 8 دستور مشخص کننده پرش است. (شکل 2-7)



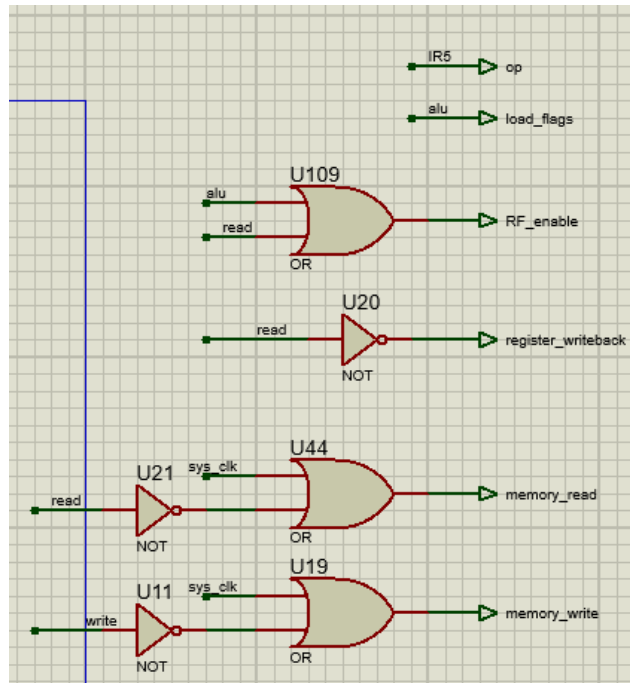
شکل 2-7. پیاده‌سازی دیکودر دستور

سپس بیت‌های آدرس مقصد و مبدا را اگر محاسبات از نوع ALU باشد طبق شکل 3-7 تعیین می‌کنیم. باید توجه کرد که بیت 5 دستور در حالت write مموری نیز مورد استفاده است.



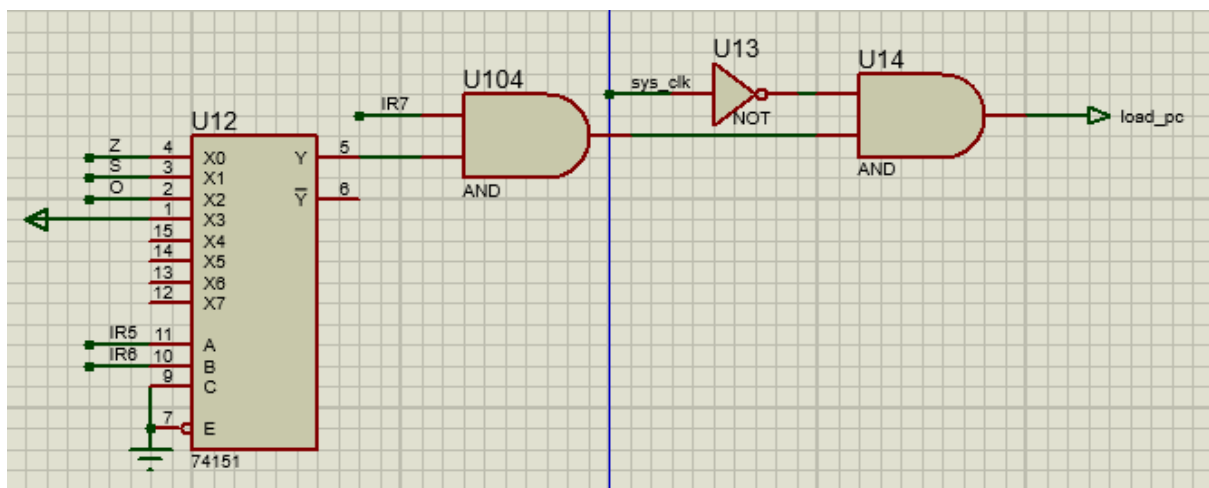
شکل 3-7. محاسبه آدرس‌های درست مبدا و مقصد

در نهایت بیت‌های کنترلی دیگر برای دستورات ارتباط با حافظه تعیین می‌شوند. (شکل 4-7)



شکل 4-7. پیاده‌سازی سیگنال‌های دستورات `alu` و دسترسی به حافظه

در نهایت سیگنال کنترلی دستور، پرش شرطی آماده می‌شود. برای این بخش ابتدا با یک دیکودر نوع پرش را تعیین می‌کنیم و سپس دستور `load_pc` را در صورت 1 شدن شرط فعال می‌کنیم. (شکل 5-7)



شکل 5-7. پیاده‌سازی سیگنال انواع دستور پرش

۳. بررسی و تست در شبیه‌سازی

نکته مهم: نمایش مقادیر میانی باس‌ها صرفاً جهت دیباگ اضافه شده‌اند و میتوان آن‌ها را حذف کرد. هرچند در نسخه تحویلی همچنان حضور دارند ولی تمام پاسخ‌های نهایی اجرای کدها یا داخل رجیسترها یا داخل حافظه ذخیره شده‌اند که در هر بخش توضیح مرتبط داده شده‌است.

برای تست 3 برنامه زیر را اجرا کرده‌ایم.

تست اول: فیبوناچی

این برنامه جملات سری فیبوناچی را محاسبه کرده و در حافظه داده گذاشته و در نهایت جمع 10 جمله اول آن را ذخیره می‌کند. کد در کل 26 بایت است و به صورت زیر تعریف می‌شود:

```
; --- Initialize Counter and Registers ---
00: Sub R0, R0 (0x20) ; Clear R0
01: Add R2, 0 (0x14) ; R2 = 0 (First term)
02: Add R3, 1 (0x1D) ; R3 = 1 (Temp calculation)
03: Add R1, 1 (0x0D) ; R1 = 1 (Second term)
04: Add R0, 1 (0x05) ; R0 = 1
05: Add R0, 1 (0x05) ; R0 = 2
06: Add R0, R0 (0x00) ; R0 = 4 (Loop counter limit)
07: Store 1 (0x61) ; Store loop counter at Mem[1]

; --- Main Loop ---
08: Sub R0, R0 (0x20) ; Clear R0
09: Add R0, R2 (0x02) ; Move R2 to R0
0A: Add R0, R1 (0x01) ; R0 = R2 + R1 (Next term)
0B: Add R1, R1 (0x09) ; Shift and move values
0C: Add R2, 0 (0x14) ; Update R2
0D: Add R3, R3 (0x1B) ; Update R3
0E: Sub R0, R0 (0x20) ; Clear R0 again
0F: Add R0, R1 (0x01) ; Temp move
10: Add R3, R3 (0x1B) ; Store value

; --- Loop Control & Branching ---
11: Load 1 (0x41) ; Load loop counter from Mem[1]
12: Sub R0, 1 (0x25) ; Decrement counter
13: JZ 16 (0x96) ; Jump to End if counter is zero
14: Store 1 (0x61) ; Store updated counter
15: Jump 08 (0xE8) ; Unconditional jump to Loop Start
```

; --- End Program ---

16: Sub R0, R0 (0x20) ; Clear R0

17: Add R0, R3 (0x03) ; Move final result to R0

18: Store 0 (0x60) ; Store final result at Mem[0]

19: Jump 19 (0xF9) ; Infinite loop to halt CPU

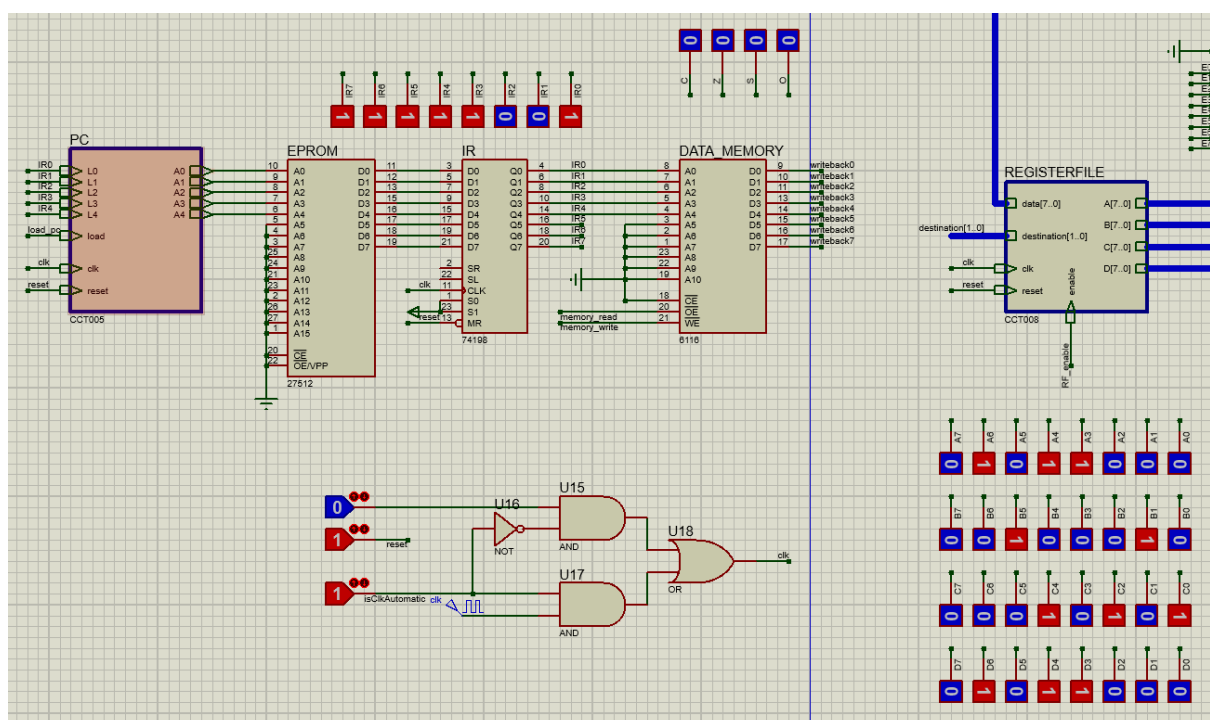
معادل هگز این برنامه داخل یک فایل هگز به نام fibonacci قرار گرفته است و به صورت زیر برای قابل فهم بودن برای برنامه ما درمی آید. یکسری بیت بیشتر از جمله checksum در این فرمت قرار دارند.

:1000000020141D0D0505006120020109141B2001AB

:0A0010001B41259661E8200360F90A

:00000001FF

نتیجه اجرا تا شماره 34, 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 که جمع مساوی 88 است که به درستی در رجیستر شماره 1 معادل باینری آن یعنی 01011000 نوشته شده است. دو عدد آخر سری یعنی 34 و 21 نیز مشخصا در رجیسترهای 1 و 2 آمده اند. (شکل 8-1)



شکل 8-1. تست فیبوناچی

تست دوم: جمع دو عدد بزرگتر از 8 بیت با رقم نقلی

این برنامه جهت بررسی صحت عملکرد بیت نقلی بین بایت‌ها و ارزیابی رفتار سخت‌افزار در مواجهه با اعداد منفی طراحی شده است.

; --- Generate Num 1 LSB: Mem[0] = -1 (0xFF) ---

00: Sub R0, R0 (0x20) ; Clear R0

01: Add R0, -1 (0x06) ; R0 = -1 (0xFF)

02: Store 0 (0x60) ; Store at Mem[0]

; --- Generate Num 1 MSB & Num 2 LSB: Mem[1] = 4, Mem[8] = 8 ---

03: Sub R0, R0 (0x20) ; Clear R0

04: Add R0, 1 (0x05) ; R0 = 1

05: Add R0, R0 (0x00) ; R0 = 2

06: Add R0, R0 (0x00) ; R0 = 4

07: Store 1 (0x61) ; Mem[1] = 4

08: Add R0, R0 (0x00) ; R0 = 8

09: Store 8 (0x68) ; Mem[8] = 8

; --- Generate Num 2 MSB: Mem[9] = -15 (0xF1) ---

0A: Sub R0, R0 (0x20) ; Clear R0

0B: Add R0, -1 (0x06) ; R0 = -1 (0xFF)

0C: Add R0, R0 (0x00) ; R0 = -2 (0xFE)

0D: Add R0, R0 (0x00) ; R0 = -4 (0xFC)

0E: Add R0, R0 (0x00) ; R0 = -8 (0xF8)

0F: Add R0, R0 (0x00) ; R0 = -16 (0xF0)

10: Add R0, 1 (0x05) ; R0 = -15 (0xF1)

11: Store 9 (0x69) ; Store at Mem[9]

; --- 16-Bit Datapath Calculation ---

; LSB Calculation: Mem[0] + Mem[8]

12: Load 0 (0x40) ; R0 = FF

13: Add R1, 0 (0x0C) ; Safe transfer to R1 (R1 = FF)

14: Load 8 (0x48) ; R0 = 08

15: Add R0, R1 (0x01) ; R0 = 08 + FF = 07 (Generates Carry = 1)

16: Store 16 (0x70) ; Result LSB

; MSB Calculation: Mem[1] + Mem[9] + Carry

17: Load 1 (0x41) ; R0 = 04

18: Add R0, C (0x07) ; R0 = 04 + 1 (Carry) = 05

19: Add R1, 0 (0x0C) ; Safe transfer to R1 (R1 = 05)

1A: Load 9 (0x49) ; R0 = F1 (-15)

1B: Add R0, R1 (0x01) ; R0 = F1 + 05 = F6

1C: Store 17 (0x71) ; Result MSB

; --- Safe Halt ---

1D: Jump 1D (0xFD) ; Infinite loop to halt CPU

1E: FF (0xFF) ; Padding

1F: FF (0xFF) ; Padding

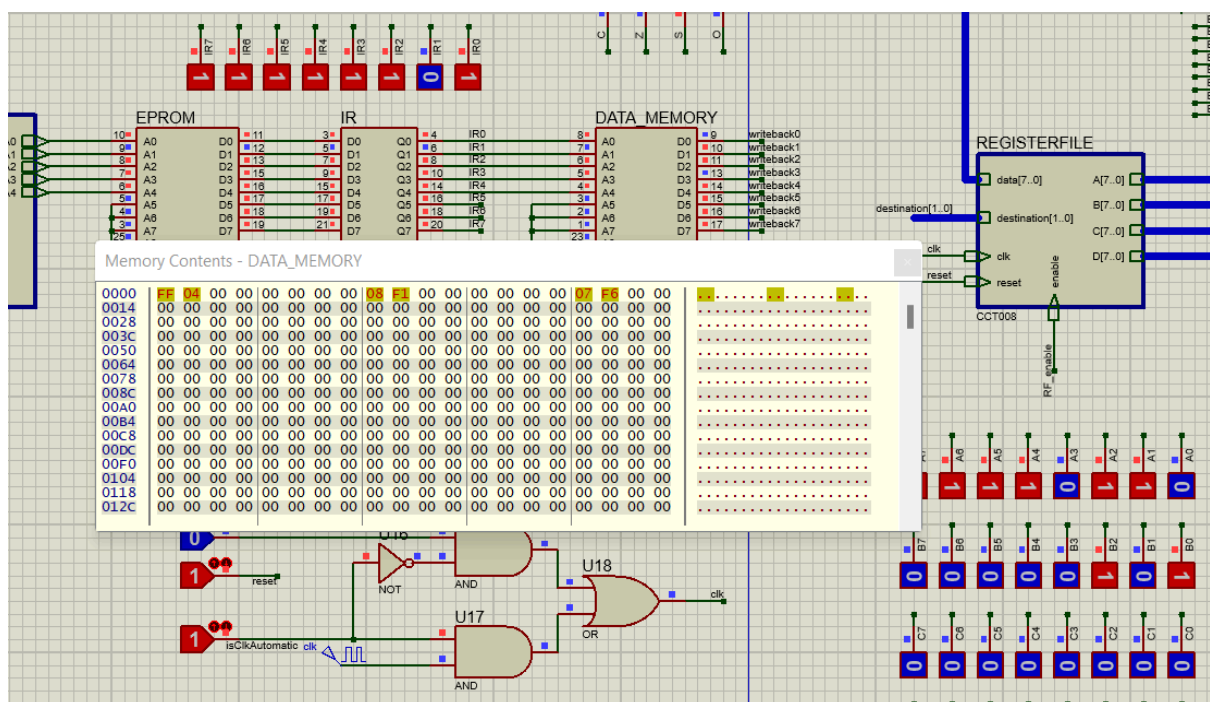
معادل هگز این برنامه داخل یک فایل هگز به نام add1 قرار گرفته است و به صورت زیر برای قابل فهم بودن برای برنامه ما درمی آید.

:1000000020066020050000610068200600000000056

:100010000569400C48017041070C490171FDFFFF63

:00000001FF

این قطعه کد، عمل جمع دو عدد ۱۶ بیتی 0x04FF (معادل ۱۲۷۹) و 0xF108 (معادل ۳۸۳۲-) را انجام می دهد. سرریز تولید شده در محاسبه بایت کم ارزش، در پرچم Carry ذخیره شده و به وسیله دستور Add R0, C به بایت پرارزش انتقال می یابد تا نتیجه نهایی منطبق بر قواعد متمم دو برابر با 0xF607 (معادل ۲۵۵۳-) در حافظه ثبت گردد (شکل ۸-۲)



شکل ۸-۲. تست جمع با رقم نقلی. پاسخ 0xF607 در خانه ۱۶ و ۱۷ حافظه مشخص هستند

تست سوم: جمع دو عدد 32 بیتی ساده

این برنامه یک عملیات جمع کامل چهار بیتی را با بهره‌گیری حداکثری از گنجایش ۳۲ بیتی EPROM و بدون استفاده از هیچ‌گونه میان‌بر محاسباتی انجام می‌دهد. به علت کم بودن جای برنامه یعنی داشتن کلا 32 دستور و این مهم که آدرس دهی غیرمستقیم نداریم، پیاده‌سازی جمع با لوپ قابل انجام نیست. نزدیک ترین چیزی که به جمع دو 64 بیتی میتوانیم برسیم برنامه زیر است.

; --- RAM Initialization ---

```
00: Sub R0, R0 (0x20) ; R0 = 00
01: Add R0, 1 (0x05) ; R0 = 01
02: Store 0 (0x60) ; Num1[0] = 01
03: Store 8 (0x68) ; Num2[0] = 01
04: Add R0, R0 (0x00) ; R0 = 02
05: Store 1 (0x61) ; Num1[1] = 02
06: Store 9 (0x69) ; Num2[1] = 02
07: Add R0, R0 (0x00) ; R0 = 04
08: Store 3 (0x63) ; Num1[3] = 04
09: Store 11 (0x6B) ; Num2[3] = 04
```

; --- Byte-by-Byte Addition ---

; Byte 0 (LSB)

```
0A: Load 0 (0x40) ; R0 = 01
0B: Add R1, 0 (0x0C) ; Safe transfer to R1
0C: Load 8 (0x48) ; R0 = 01
0D: Add R0, R1 (0x01) ; R0 = 01 + 01 = 02
0E: Store 16 (0x70) ; Result[0] = 02
```

; Byte 1

```
0F: Load 1 (0x41) ; R0 = 02
10: Add R1, 0 (0x0C) ; Safe transfer to R1
11: Load 9 (0x49) ; R0 = 02
12: Add R0, R1 (0x01) ; R0 = 02 + 02 = 04
13: Store 17 (0x71) ; Result[1] = 04
```

; Byte 2 (Explicit calculation of 00 bytes)

```
14: Load 2 (0x42) ; R0 = 00
15: Add R1, 0 (0x0C) ; Safe transfer to R1
16: Load 10 (0x4A) ; R0 = 00
17: Add R0, R1 (0x01) ; R0 = 00 + 00 = 00
18: Store 18 (0x72) ; Result[2] = 00
```

; Byte 3 (MSB)

```
19: Load 3 (0x43) ; R0 = 04
```

شکل 8-3. تست جمع دو رقم 32 بیتی